

Computational and statistical learning

Lab: Stochastic Gradient Methods

Giorgia Franchini
giorgia.franchini@unimore.it

Università degli Studi di Modena e Reggio Emilia

28 April 2025

Optimization problem in machine learning

The following optimization problem, which minimizes the sum of cost functions over samples from a finite training set composed by sample data $x_i \in \mathbb{R}^d$ and class label $y_i \in \{\pm 1\}$ for $i \in \{1 \dots n\}$, appears frequently in machine learning:

$$\min F(\omega) \equiv \frac{1}{n} \sum_{i=1}^n f_i(\omega), \quad (1)$$

where n is the sample size, and each $f_i : \mathbb{R}^d \rightarrow \mathbb{R}$ is the cost function corresponding to a training set element.

For example in the logistic regression case we have:

$$f_i(\omega) = \log \left[1 + \exp(-y_i x_i^T \omega) \right]$$

We are interested in finding ω that minimizes (1).

Stochastic Gradient (SG)

- For given ω , computing $F(\omega)$ and $\nabla F(\omega)$ is prohibited, due to the large size of the training set;
- when n is large, Stochastic Gradient (SG) method and its variants have been the main approaches for solving (1);
- in the k – *th* iteration of SGD, a random index of a training sample i_k is chosen from $\{1, 2, \dots, n\}$ and the iterate ω_k is updated by

$$\omega_{k+1} = \omega_k - \alpha_k \nabla f_{i_k}(\omega_k)$$

where $\nabla f_{i_k}(\omega_k)$ denotes the gradient of the i_k – *th* component function at ω_k , and $\alpha_k > 0$ is the step size or learning rate.

A numerical experiment: the test problem

- Logistic regression with l_2 – *norm* regularization:

$$\min_x F(x) = \frac{1}{n} \sum_{i=1}^n \log \left[1 + \exp(-y_i x_i^T \omega) \right] + \frac{\lambda}{2} \|\omega\|_2^2$$

where $x_i \in \mathbb{R}^d$ and $y_i \in \{\pm 1\}$ are the feature vectors and class labels of the i – *th* sample, respectively, and $\lambda > 0$ is a regularization parameter;

- database: MNIST 8 and 9 digits, dimension: 11800×784 .

MNIST



A numerical experiment: the algorithms

Compare:

- Full gradient BB rules:
 - BB1 full gradient.
 - BB2 full gradient.
- Stochastic:
 - SG, Stochastic Gradient;
 - SG with MOMENTUM;
 - AdaM: stochastic gradient method based on adaptive moment estimation.
- Behaviour with respect to the epochs: one epoch = 11800 SG steps.

Deterministic cases

$$\eta_k^{BB1} = \frac{s_{k-1}^T s_{k-1}}{s_{k-1}^T v_{k-1}} \quad \eta_k^{BB2} = \frac{s_{k-1}^T v_{k-1}}{v_{k-1}^T v_{k-1}}$$

where $s_{k-1} = \omega_k - \omega_{k-1}$ and $v_{k-1} = \nabla F(\omega_k) - \nabla F(\omega_{k-1})$ with

$$\nabla F(\omega) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\omega).$$

Algorithm Stochastic Gradient (SG) Method

- 1: Choose an initial iterate ω_0 .
 - 2: **for** $k = 0, 1, \dots$ **do**
 - 3: Generate a realisation of the random variable ξ_{n_k} .
 - 4: Compute a stochastic vector $g(\omega_k, \xi_{n_k})$.
 - 5: Choose a learning rate $\alpha_k > 0$.
 - 6: Set the new iterate as $\omega_{k+1} \leftarrow \omega_k - \alpha_k g(\omega_k, \xi_{n_k})$.
 - 7: **end for**
-

Momentum

Algorithm Momentum

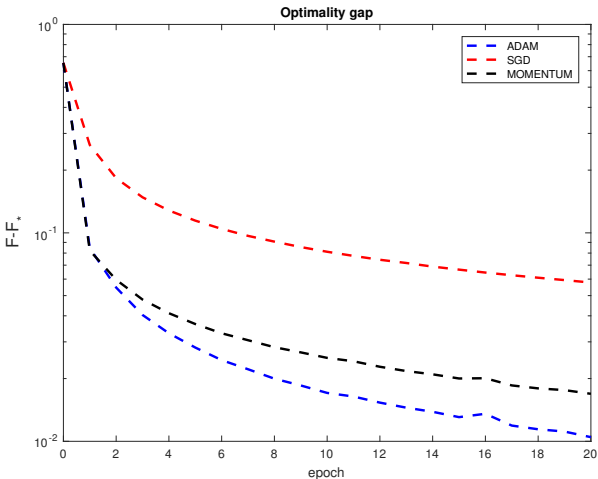
- 1: Choose *maxit*, $\alpha, \beta \in [0, 1)$, ω_0 ;
 - 2: initialize $m_0 \leftarrow 0$, $k \leftarrow 0$
 - 3: **for** $k \in \{0, \dots, \text{maxit}\}$ **do**
 - 4: $k \leftarrow k + 1$
 - 5: $g_k \leftarrow \nabla f_{i_k}(\omega_{k-1})$
 - 6: $m_k \leftarrow \beta \cdot m_{k-1} + g_k$
 - 7: $\omega_k \leftarrow \omega_{k-1} - \alpha \cdot m_k$
 - 8: **end for**
 - 9: Result: ω_k
-

AdaM

Algorithm Adam

- 1: Choose α , $\hat{\epsilon}$, β_1 and $\beta_2 \in [0, 1)$, ω_0 ;
 - 2: initialize $m_0 \leftarrow 0$, $v_0 \leftarrow 0$, $k \leftarrow 0$
 - 3: **for** $k = 0, 1, \dots$ **do**
 - 4: $k \leftarrow k + 1$
 - 5: $g_k \leftarrow \nabla f_{i_k}(\omega_{k+1})$
 - 6: $m_k \leftarrow \beta_1 \cdot m_{k-1} + (1 - \beta_1) \cdot g_k$
 - 7: $v_k \leftarrow \beta_2 \cdot v_{k-1} + (1 - \beta_2) \cdot g_k^2$
 - 8: $\alpha_k = \alpha \frac{\sqrt{1 - \beta_2^k}}{(1 - \beta_1^k)}$
 - 9: $\omega_k \leftarrow \omega_{k+1} - \alpha_k \cdot m_k / (\sqrt{v_k} + \hat{\epsilon})$
 - 10: **end for**
-

Comparison between different SGD types



The minimization problem

We built linear classifiers corresponding to three different loss functions; in all cases, a regularization term was added to avoid overfitting. Thus the minimization problem has the form

$$\min_{\omega} F_n(\omega) + \frac{\lambda}{2} \|\omega\|_2^2,$$

where $\lambda > 0$ is a regularization parameter, $x_i \in \mathbb{R}^d$ and $y_i \in \{1, -1\}$ are the feature vector and the class label of the i -th sample, respectively.

Logistic regression

$$F_n(\omega) = \frac{1}{n} \sum_{i=1}^n \log \left[1 + e^{(-y_i x_i^T \omega)} \right];$$

where $x_i \in \mathbb{R}^d$ and $y_i \in \{\pm 1\}$ are the feature vectors and class labels of the i -th sample, respectively, and $\lambda > 0$ is a regularization parameter.

Its gradient is given by

$$\nabla F(\omega) = \frac{1}{n} \sum_{i=1}^n \frac{\exp(-x_i y_i^T \omega) (-y_i x_i^T)}{[1 + \exp(-y_i x_i^T \omega)]};$$

Accuracy

Hyperplane:

$$Y = \omega^T X$$

Predicted output

$$\bar{y}(X) = \text{sign}(\omega^T X)$$

Accuracy:

$$\text{acc} = \frac{n - \text{nnz}(\text{sign}(Y) - \text{labels}^T)}{n}$$